

Résolution des équations différentielles

December 8, 2021

```
[1]: %pylab inline
```

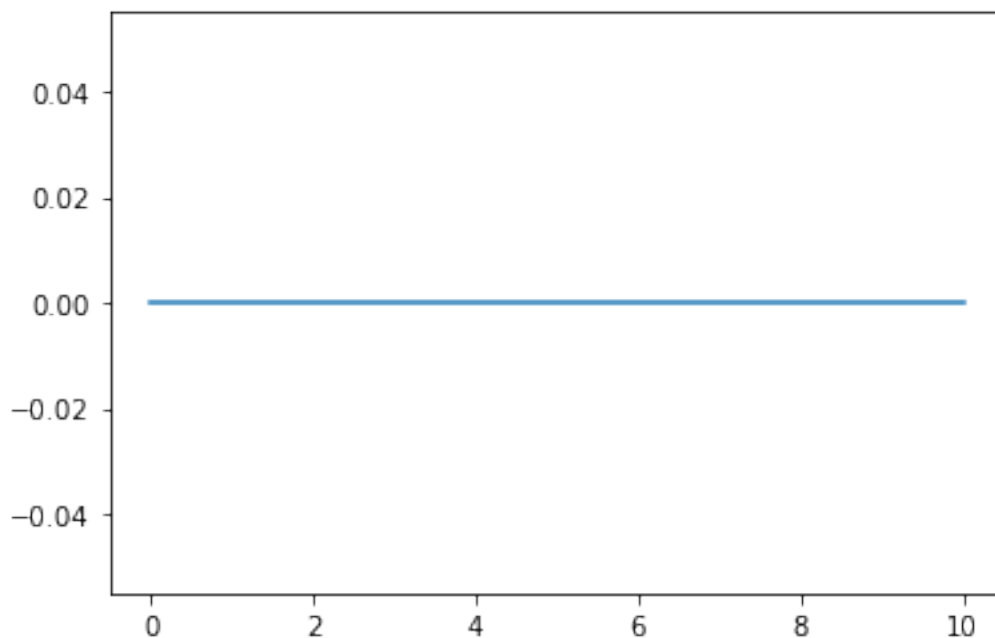
Populating the interactive namespace from numpy and matplotlib

1 Equation différentielle

1.1 Second membre non lipschitzien

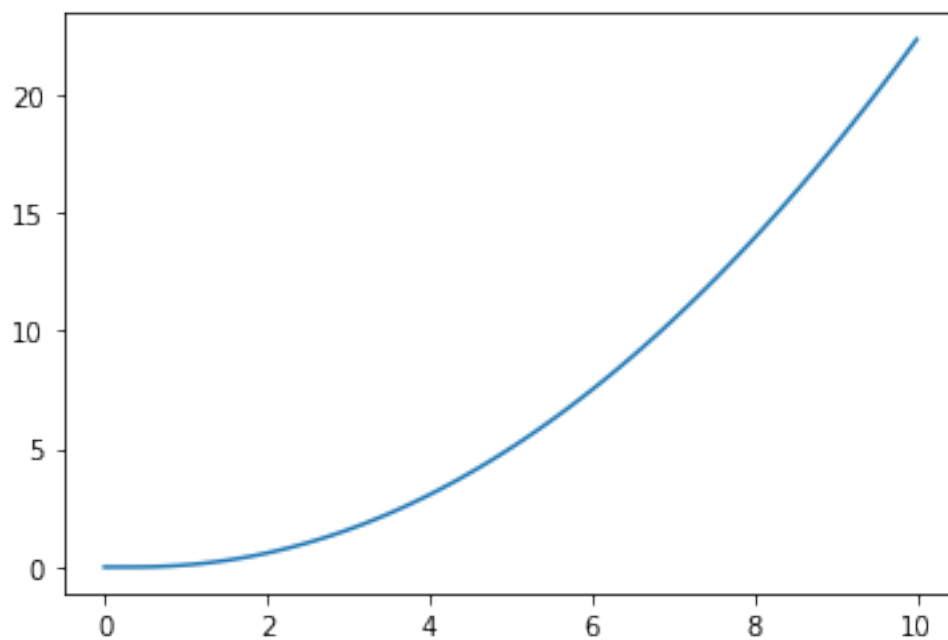
```
[20]: def euler_explicite(f,ti,tf,x0,N):  
    h=(tf-ti)/N  
    t,x=linspace(ti,tf,N+1), []  
    x.append(x0)  
    for n in range(N):  
        x.append(x[n]+h*f(t[n],x[n]))  
    return t,x
```

```
[21]: def f(t,x):  
    return sqrt(x)  
x0=[0]  
t,x=euler_explicite(f,0,10,x0,100)  
plot(t,x)  
show()
```



En remplaçant la valeur de la donnée initiale par l'évaluation numérique de $|\sqrt{(2)^2 - 2}|$ on remarque que la résolution approchée est complètement modifiée.

```
[7]: x0=array([abs(sqrt(2.）**2-2.)])
t,x=euler_explicite(f,0.,10.,x0,100)
plot(t,x)
show()
```

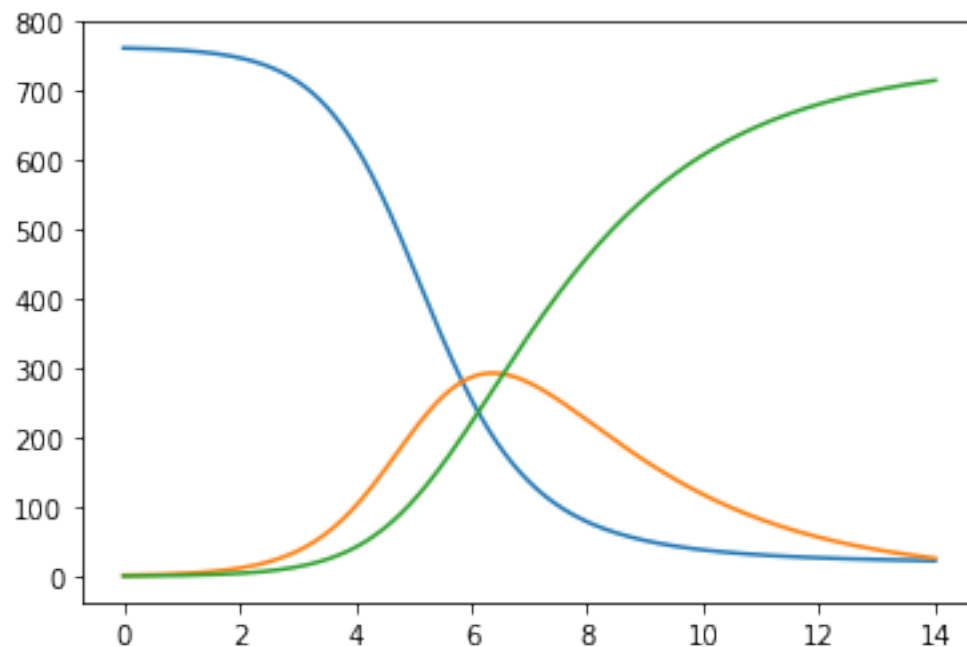


En raison des erreurs d'arrondi ayant lieu lors des calculs en arithmétique en précision finie, la valeur initiale utilisée dans cette question peut être vue comme une petite perturbation de la valeur initiale précédemment employée. Bien que ce soit une perturbation minime (on pourra afficher la valeur numérique de la variable x_0 pour s'en assurer), la solution approchée fournie par la méthode est très différente de celle précédemment calculée. La fonction f considérée n'étant pas lipschitzienne, la stabilité de la méthode à l'égard des perturbations n'est en effet pas garantie.

1.2 Modele SIR

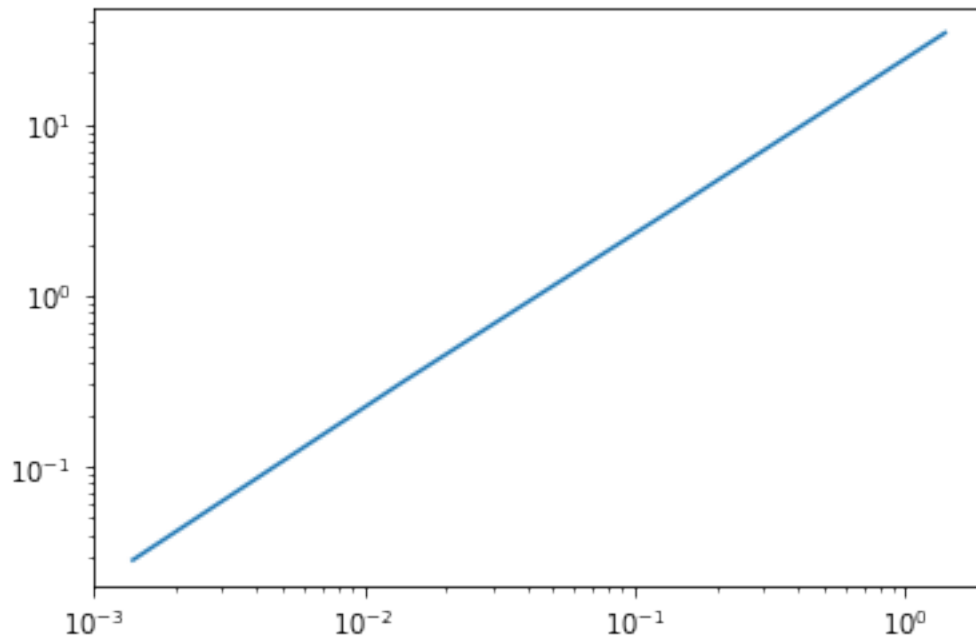
```
[26]: r,a=0.00218,0.44036
def f(t,x) :
    return array([-r*x[0]*x[1],r*x[0]*x[1]-a*x[1],a*x[1]])
```

```
[28]: x0=array([762.,1.,0])
t,x=euler_explicite(f,0.,14.,x0,10**5)
plot(t,x)
show()
```



En prenant comme solution de référence celle obtenue avec $N = 105$ pas de discrétisation, nous représentons (avec une échelle logarithmique) les erreurs au temps T trouvées en utilisant successivement $N = 10, 10^2, \dots, 10^4$ pas de discrétisation.

```
[31]: t,x=euler_explicite(f,0.,14.,x0,10**5)
xref,err,h=x[-1],[],[]
for i in range(4) :
    N=10**(i+1)
    h.append(14/N)
    t,x=euler_explicite(f,0.,14.,x0,N)
    err.append(norm(xref-x[-1]))
loglog(h,err)
show()
```



Pour déterminer l'ordre de la méthode, il faut revenir à la définition de cette notion. L'erreur globale d'une méthode d'ordre p , avec p un entier, décroît avec le pas de discrétisation h comme $O(h^p)$ et l'on a donc, pour h suffisamment petit,

$$\text{erreur} \propto Ch^p,$$

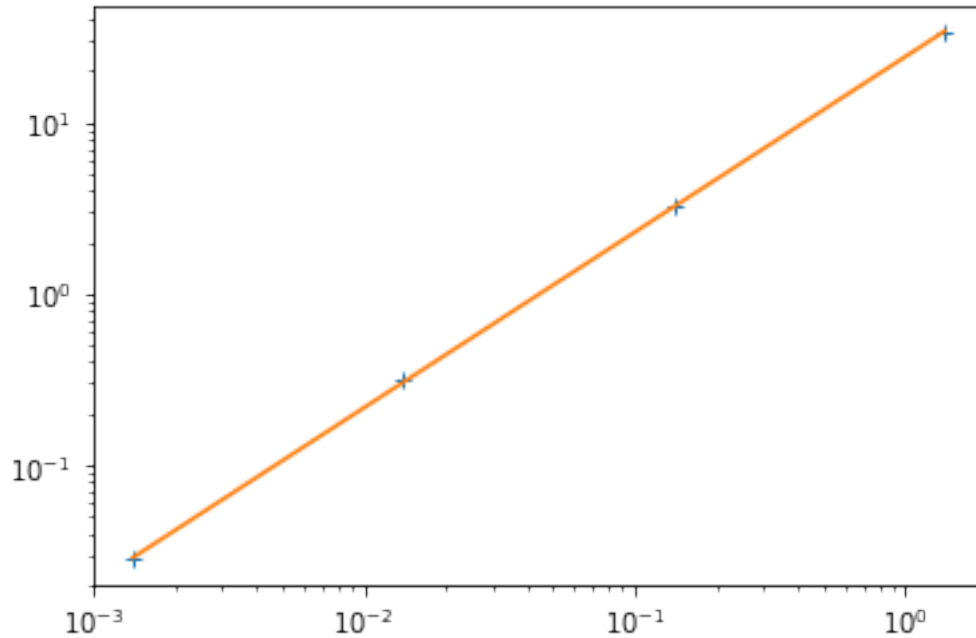
avec C une constante. En prenant le logarithme de cette relation, il vient

$$\ln(\text{erreur}) = p \ln(h) + \ln(C).$$

Il suffit donc, à partir des valeurs de l'erreur approchée obtenue avec les différents choix de pas, déterminer la pente de la droite obtenue par une régression linéaire au moyen de la fonction `polyfit`.

```
[32]: slope=polyfit(log(h),log(err),1)
loglog(h,err,'+',h,exp(slope[1])*(h**(slope[0])))
print("L'ordre de convergence effectif vaut",slope[0])
```

L'ordre de convergence effectif vaut 1.0241140156673936

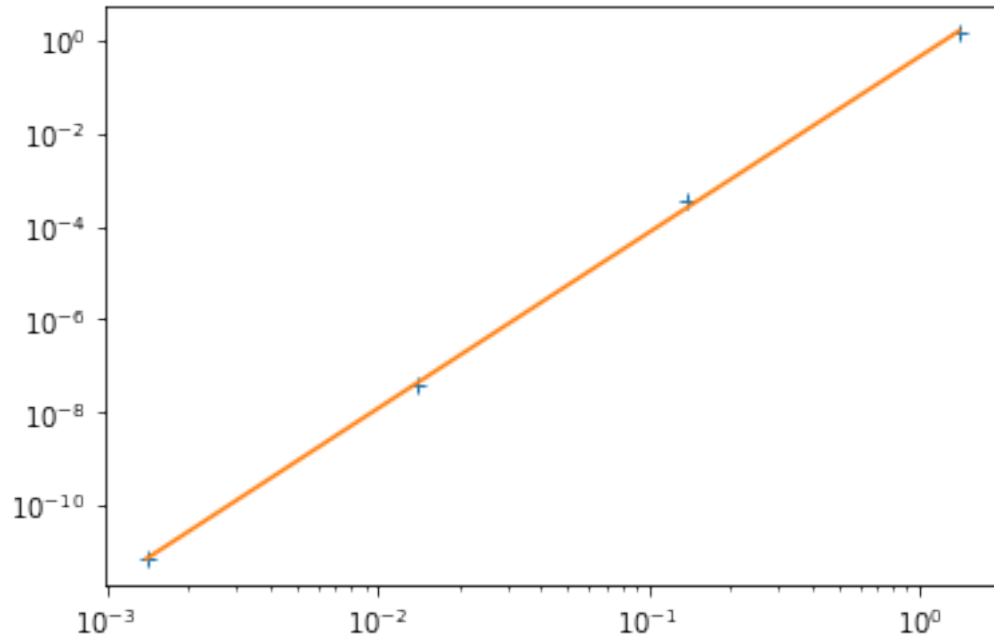


1.3 Méthode de runge-kutta

```
[33]: def rk4(f,ti,tf,x0,N):
    h=(tf-ti)/N
    t,x=linspace(ti,tf,N+1),[]
    x.append(x0)
    for n in range(N):
        k1=f(t[n],x[n])
        k2=f(t[n]+0.5*h,x[n]+0.5*h*k1)
        k3=f(t[n]+0.5*h,x[n]+0.5*h*k2)
        k4=f(t[n]+h,x[n]+h*k3)
        x.append(x[n]+h*(k1+2.*(k2+k3)+k4)/6.)
    return t,x
```

```
[34]: t,x=rk4(f,0.,14.,x0,10**5)
xref,err,h=x[-1],[],[]
for i in range(4):
    N=10**(i+1)
    h.append(14/N)
    t,x=rk4(f,0.,14.,x0,N)
    err.append(norm(xref-x[-1]))
slope=polyfit(log(h),log(err),1)
loglog(h,err,'+',h,exp(slope[1])*(h**(slope[0])))
print("L'ordre de convergence effectif vaut",slope[0])
```

L'ordre de convergence effectif vaut 3.7856103572034923



1.4 Exemple de problème raide

On cherche à approcher numériquement la solution du problème de Cauchy

$$x'(t) = -50(x(t) - \cos(t)), \quad t > 0, \quad \text{et } x(0) = 0,$$

sur l'intervalle $[0, 2]$.

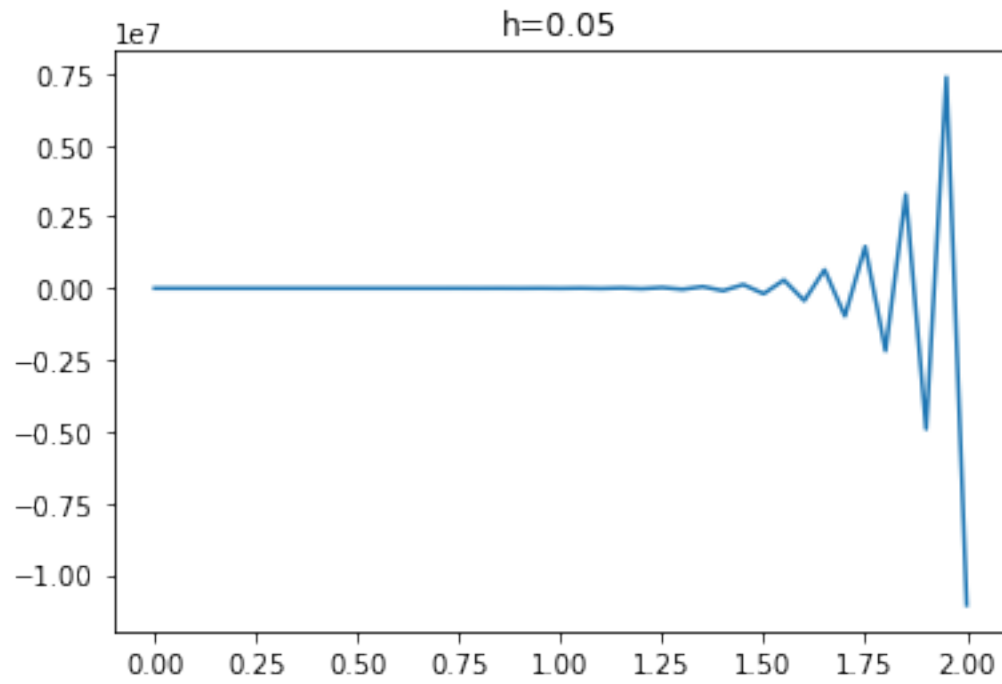
On utilise la méthode d'Euler explicite pour résoudre le problème avec un pas de discrétisation de longueur constante, successivement choisie:

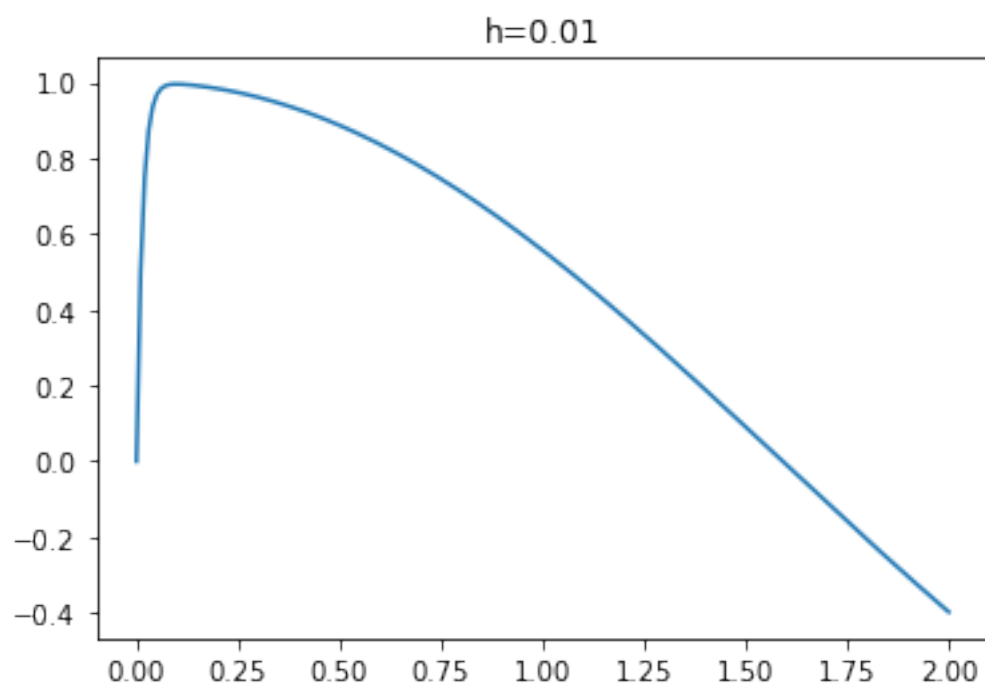
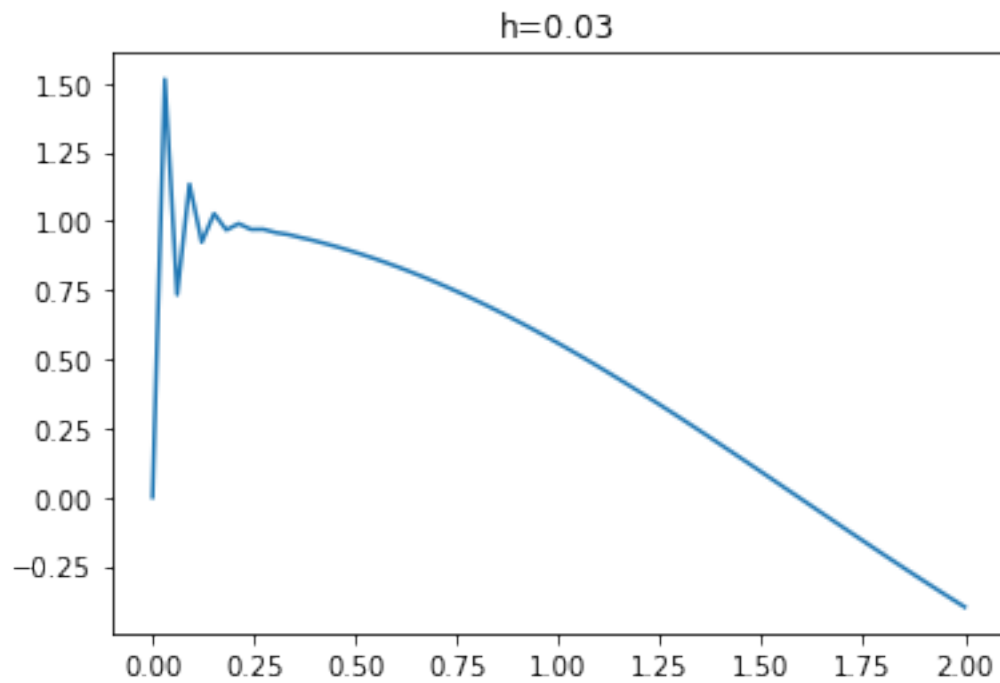
- strictement supérieure à 0,04,
- comprise entre 0,02 et 0,04,
- strictement inférieure à 0,02.

```
[37]: def f(t,x): return -50*(x-cos(t))
x0=array([0.])
figure()
t,x=euler_explicite(f,0,2,x0,int(2/0.05))
plot(t,x)
title("h=0.05")
figure()
t,x=euler_explicite(f,0,2,x0,int(2/0.03))
plot(t,x)
title("h=0.03")
figure()
```

```
t,x=euler_explicite(f,0,2,x0,int(2/0.01))  
plot(t,x)  
title("h=0.01")
```

[37]: Text(0.5, 1.0, 'h=0.01')





Pour un pas de longueur $h=0.005$, on observe un important phénomène d'instabilité numérique dans l'approximation calculée. Avec un pas de longueur $h=0.03$, le phénomène est moins marqué,

mais clairement visible aux premiers instants de la simulation. Enfin, avec un pas de longueur $h=0.01$, l'approximation obtenue ne présente plus d'instabilité numérique.

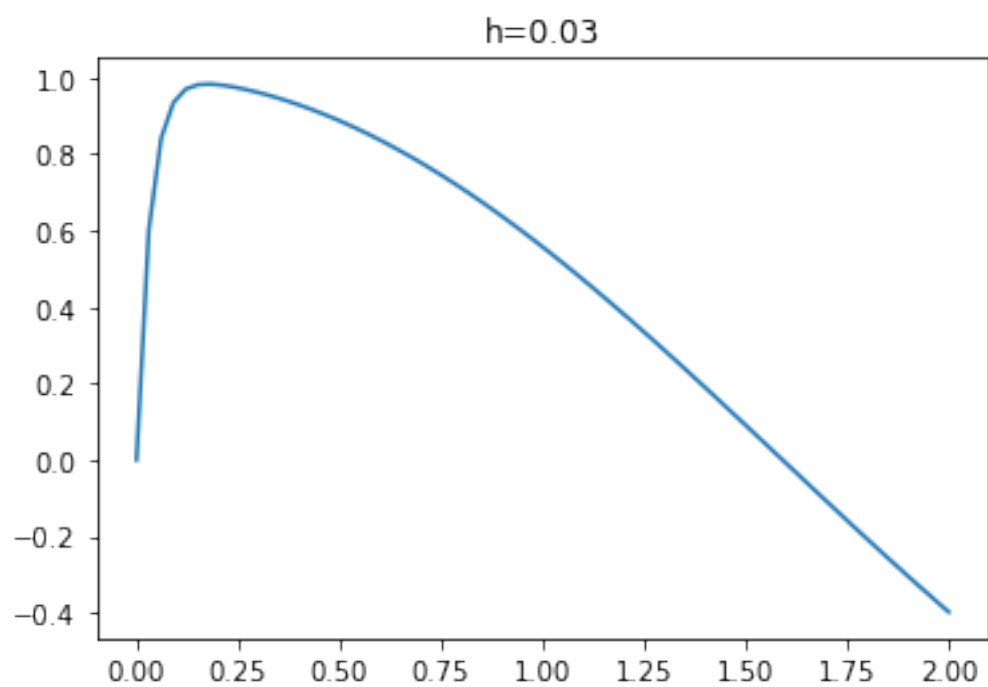
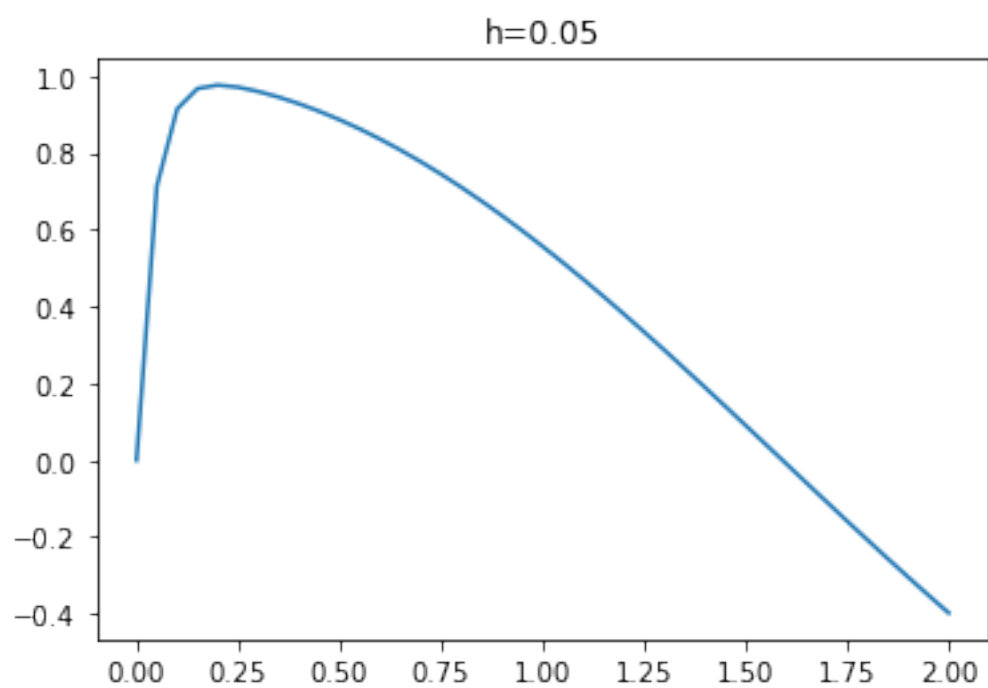
On qualifie de raide tout système d'équations différentielles ordinaires pour lequel une méthode de résolution numérique explicite présente des problèmes d'instabilité numérique, à moins que la longueur du pas de discrétisation ne soit extrêmement petite (ou, pour être plus précis, beaucoup plus petite que ne l'exigerait en principe la précision demandée lorsqu'on utilise un mécanisme d'adaptation du pas). Les simulations effectuées indiquent ici qu'on est en présence d'une équation de ce type.

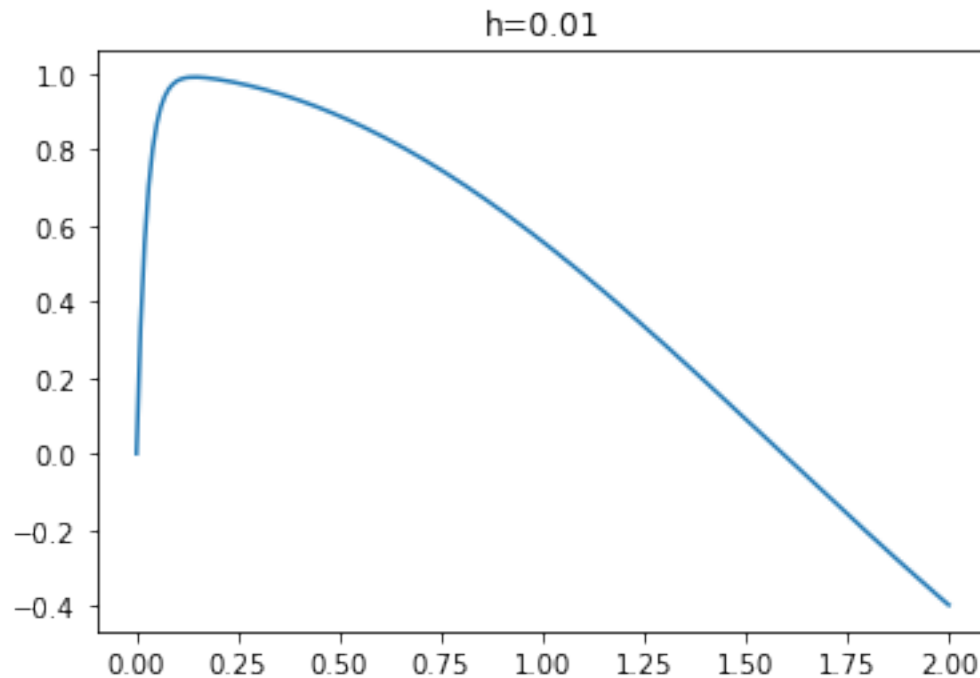
Effectuons les calculs précédents avec la méthode d'Euler implicite. Pour cela, écrire une fonction `euler_implicite` dans laquelle la méthode de Newton-Raphson est employée pour résoudre le système d'équations non linéaires (on ajoutera dans les arguments d'entrée la fonction donnant la dérivée par rapport à x de la fonction f définissant le membre de droite de l'équation différentielle, ainsi que des paramètres d'arrêt pour la méthode de Newton-Raphson).

```
[38]: def euler_implicite(f,df,ti,tf,x0,N,tol=1e-6,maxiter=50):
    h=(tf-ti)/N
    t,x=linspace(ti,tf,N+1),[]
    x.append(x0)
    neq=len(x0)
    for n in range(N):
        xnew=x[n]
        for j in range(maxiter):
            rhs=(xnew-x[n])-h*f(t[n]+h,xnew)
            jac=eye(neq)-h*df(t[n]+h,xnew)
            inc=solve(jac,rhs)
            xnew=xnew-inc
            if norm(inc)<tol:
                break
        x.append(xnew)
    return t,x
```

```
[39]: def df(t,x): return -50
x0=array([0.])
figure()
t,x=euler_implicite(f,df,0,2,x0,int(2/0.05))
plot(t,x)
title("h=0.05")
figure()
t,x=euler_implicite(f,df,0,2,x0,int(2/0.03))
plot(t,x)
title("h=0.03")
figure()
t,x=euler_implicite(f,df,0,2,x0,int(2/0.01))
plot(t,x)
title("h=0.01")
```

[39]: Text(0.5, 1.0, 'h=0.01')





La méthode d'Euler implicite possède un domaine de stabilité absolue non borné, qui contient l'ensemble des nombres complexes dont la partie réelle est strictement négative. On dit qu'elle est A-stable. Une méthode A-stable n'est pas sujette aux problèmes d'instabilité lorsqu'on l'utilise pour la résolution d'une équation raide, comme on peut le constater ici.

1.5 Résolution par `scipy.integrate`

```
[ ]: 
```

```
[ ]: 
```